



UNIVERSIDAD AUTÓNOMA DE GUADALAJARA

Licenciatura en Actuaría y Ciencia de Datos

Materia: Minería de Datos y Ciencia de Datos

Mini Proyecto: Sistema de Sentiment Analysis con NLP

Amazon Reviews: Kindle Store Category

Profesor: M.C. Eduardo Francisco Sánchez Ocampo

Integrantes: Pablo García Espejo

Diego Arath Franco Herrera

Miguel Ángel Guerrero Alvarez

Manuel de la Torre

Giulian Thibaut Elías-Libera

19 de febrero de 2026

Índice

Resumen de cumplimiento	1
1. Reporte ejecutivo	2
1.1. Objetivo del proyecto	2
1.2. Dataset utilizado	2
1.3. Resultado principal	3
1.4. Conclusiones ejecutivas	3
2. Memoria técnica	5
2.1. Descripción general del sistema	5
2.2. Estructura del repositorio	5
2.3. Preprocesamiento NLP	6
2.4. Representación vectorial del texto	7
2.4.1. Bag of Words	7
2.4.2. TF-IDF	7
2.5. Modelos implementados	7
2.6. Experimentos realizados	8
3. Evidencia de ejecución real	9
4. Resultados obtenidos	11
4.1. Comparación de F1-score	13
4.2. Comparación de tiempo de entrenamiento	13
5. Matrices de confusión	15
5.1. Bag of Words con lemmatization	15
5.2. Bag of Words con stemming	15
5.3. TF-IDF con lemmatization	16

5.4. TF-IDF con stemming	16
5.5. Interpretación general de las matrices	16
6. Análisis comparativo	17
6.1. Bag of Words vs. TF-IDF	17
6.2. Stemming vs. lemmatization	17
6.3. Comparación de modelos	17
6.4. Problema de desbalance de clases	17
7. Validación y pruebas automáticas	17
8. Cómo ejecutar el proyecto	18
8.1. Crear ambiente virtual	18
8.2. Instalar dependencias	18
8.3. Ejecutar con dataset real	19
8.4. Ejecutar con dataset de muestra	19
9. Documentación del uso de IA	19
9.1. Errores encontrados y correcciones	20
10.Conclusiones	20
11.Referencias	21

Resumen de cumplimiento

Este reporte documenta un sistema completo de *Sentiment Analysis* con técnicas de *Natural Language Processing* aplicado al dataset *Amazon Reviews: Kindle Store Category*. El proyecto incluye carga de datos, limpieza textual, tokenización, eliminación de *stopwords*, stemming, lemmatization, representación vectorial con Bag of Words y TF-IDF, entrenamiento de modelos de clasificación, evaluación comparativa, matrices de confusión y evidencias reales de ejecución.

Cuadro 1: Mapa de requerimientos del proyecto y ubicación en el reporte

Requerimiento	Implementación en el reporte
Dataset y planteamiento del problema	Se utiliza el dataset <i>Amazon Reviews: Kindle Store Category</i> y se plantea una clasificación binaria positivo/negativo.
Preprocesamiento NLP	Limpieza de texto, tokenización, eliminación de <i>stopwords</i> , stemming y lemmatization.
Vectorización	Comparación entre Bag of Words y TF-IDF.
Modelos supervisados	Naive Bayes, Logistic Regression y SVM lineal.
Experimentos comparativos	Doce combinaciones entre procesamiento, vectorización y modelo.
Métricas de evaluación	Train accuracy, test accuracy, precision, recall, F1-score, tiempo de entrenamiento y matrices de confusión.
Evidencia real	Se incluyen las capturas reales de ejecución y las gráficas con los nombres de archivo compartidos.
Documentación del uso de IA	Se describe el apoyo de IA y las validaciones realizadas por el equipo.

Nota importante sobre las imágenes. Las figuras incluidas en este documento usan los nombres reales observados en la carpeta de resultados: `comparacion_f1_score.png`, `comparacion_tiempo_entrenamiento.png`, las doce matrices `confusion_matrix_...png` y las capturas `evidencia_ejecucion_resultados_reales_1.png` y `evidencia_ejecucion_resultados_reales_2.png`. Para compilar sin cambios, coloque esos archivos en la misma carpeta del `.tex` o dentro de `outputs/figures/`.

1. Reporte ejecutivo

1.1. Objetivo del proyecto

El objetivo del mini proyecto es construir un sistema completo de *Sentiment Analysis* utilizando técnicas de *Natural Language Processing*. El sistema procesa reseñas escritas por usuarios, transforma el texto en representaciones numéricas y entrena modelos de *Machine Learning* para clasificar automáticamente el sentimiento expresado en una reseña como positivo o negativo.

1.2. Dataset utilizado

El dataset utilizado fue *Amazon Reviews: Kindle Store Category*, disponible en Kaggle. Contiene reseñas de usuarios sobre productos de la categoría Kindle Store de Amazon. Para este proyecto, la variable de calificación se transformó en una etiqueta binaria:

Cuadro 2: Transformación de calificaciones a etiquetas binarias

Calificación original	Tratamiento	Etiqueta final
4 o 5	Reseña positiva	1
1 o 2	Reseña negativa	0
3	Se elimina por neutralidad	No aplica

La ejecución real se realizó con el archivo:

```
data/raw/kindle_reviews.csv
```

Después de eliminar reseñas neutrales, el conjunto final usado en la ejecución tuvo **886,404 reseñas**, distribuidas de la siguiente forma:

Cuadro 3: Distribución de clases en la ejecución real

Clase	Cantidad
Positivo	829,256
Negativo	57,148
Total	886,404

El dataset está desbalanceado porque las reseñas positivas son mucho más frecuentes que las negativas. Por esta razón, el F1-score se considera una métrica clave junto con la exactitud.

1.3. Resultado principal

La mejor configuración fue **TF-IDF + lemmatization + SVM lineal**. Esta combinación obtuvo:

Cuadro 4: Mejor configuración obtenida en la ejecución real

Campo	Valor
Representación	TF-IDF
Procesamiento	Lemmatization
Modelo	SVM lineal
Train Accuracy	0.9732
Test Accuracy	0.9711
Precision	0.9779
Recall	0.9915
F1-score	0.9847
Tiempo	139.2664 s

Este resultado muestra que el modelo clasifica correctamente la mayoría de las reseñas y mantiene buen balance entre *precision* y *recall*. El desempeño alto se explica por la combinación de una representación TF-IDF, que pondera mejor las palabras importantes, con un modelo SVM lineal, que funciona bien en espacios dispersos y de alta dimensión.

1.4. Conclusiones ejecutivas

- El sistema implementa un flujo completo de NLP: carga de datos, limpieza, tokenización, eliminación de *stopwords*, stemming, lemmatization, vectorización, entrenamiento y evaluación.
- TF-IDF fue la representación más fuerte para el mejor resultado general, porque reduce el peso de palabras demasiado comunes y destaca términos más informativos.
- SVM lineal fue el modelo con mejor F1-score, lo que confirma que es adecuado para clasificación de texto con vectores dispersos.
- El desbalance de clases es el principal reto: existen 829,256 reseñas positivas frente a 57,148 negativas. Por eso, accuracy no basta y se debe revisar F1-score, precision y recall.

- Las matrices de confusión y las capturas de terminal incluidas documentan la ejecución real del pipeline.

2. Memoria técnica

2.1. Descripción general del sistema

El sistema sigue el siguiente flujo de trabajo:

1. Carga el dataset de reseñas.
2. Detecta la columna de texto y la columna de calificación.
3. Convierte las calificaciones en etiquetas binarias.
4. Elimina reseñas neutrales con calificación 3.
5. Limpia el texto.
6. Tokeniza las reseñas.
7. Elimina *stopwords*.
8. Aplica stemming o lemmatization.
9. Convierte texto a vectores usando Bag of Words o TF-IDF.
10. Divide los datos en entrenamiento y prueba.
11. Entrena Naive Bayes, Logistic Regression y SVM lineal.
12. Calcula métricas y guarda reportes, gráficas y matrices de confusión.

2.2. Estructura del repositorio

La estructura del proyecto fue organizada para separar código fuente, datos, reportes, pruebas y resultados:

```
sentiment-analysis-kindle-nlp/
|-- .github/
|   '-- workflows/
|       '-- ci.yml
|-- data/
|   |-- raw/
|   |   '-- .gitkeep
|   '-- sample/
|       '-- kindle_reviews_sample.csv
|-- docs/
|   |-- evidencias/
|   |-- presentacion/
|   |   |-- presentacion_sentiment_analysis_beamer.tex
|   |   '-- presentacion_sentiment_analysis.pdf
|   '-- reporte/
|       |-- reporte_sentiment_analysis.tex
|       '-- reporte_sentiment_analysis.pdf
```



```

|-- outputs/
|   |-- figures/
|   |   |-- comparacion_f1_score.png
|   |   |-- comparacion_tiempo_entrenamiento.png
|   |   |-- evidencia_ejecucion_resultados_reales_1.png
|   |   |-- evidencia_ejecucion_resultados_reales_2.png
|   |-- reports/
|   |   |-- classification_reports.txt
|   |   |-- resultados_comparativos.csv
|   |   |-- resultados_comparativos.md
|-- referencia_clase/
|   |-- nlp_base_clase.py
|-- src/
|   |-- __init__.py
|   |-- config.py
|   |-- data_loader.py
|   |-- evaluate.py
|   |-- main.py
|   |-- modeling.py
|   |-- predict.py
|   |-- preprocessing.py
|-- tests/
|   |-- test_data_loader.py
|   |-- test_preprocessing.py
|-- .gitignore
|-- LICENSE
|-- MiniProyecto3_Sentiment_Analysis_NLP.ipynb
|-- README.md
|-- requirements.txt

```

2.3. Preprocesamiento NLP

El archivo principal del preprocesamiento es `src/preprocessing.py`. La finalidad de esta etapa es transformar el texto crudo en una versión limpia, consistente y útil para los algoritmos de clasificación.

Los pasos implementados son:

- Conversión de texto a minúsculas.
- Eliminación de URLs.
- Eliminación de etiquetas HTML.
- Eliminación de signos de puntuación.
- Eliminación de números.
- Eliminación de caracteres especiales.
- Eliminación de espacios repetidos.

- Tokenización.
- Eliminación de *stopwords*.
- Aplicación de stemming.
- Aplicación de lemmatization.

El stemming reduce las palabras a una raíz aproximada. Por ejemplo, distintas variaciones de una palabra pueden quedar en una forma común, aunque no siempre lingüísticamente perfecta. La lemmatization busca una forma base más limpia y cercana al significado original de la palabra. En clasificación de texto, ambas técnicas permiten reducir la dimensionalidad del vocabulario.

2.4. Representación vectorial del texto

2.4.1. Bag of Words

Bag of Words representa cada reseña mediante la frecuencia de las palabras del vocabulario. Es una técnica simple, interpretable y útil como línea base. Su principal limitación es que no conserva el orden de las palabras ni captura relaciones semánticas profundas.

2.4.2. TF-IDF

TF-IDF asigna peso a una palabra según su frecuencia dentro de un documento y su rareza dentro del corpus. De esta forma, palabras muy comunes reciben menor peso, mientras que términos más informativos reciben mayor importancia. Esta representación suele funcionar bien en clasificación de documentos y reseñas.

2.5. Modelos implementados

Se entrenaron tres modelos supervisados:

Cuadro 5: Modelos de clasificación utilizados

Modelo	Descripción
Naive Bayes	Modelo probabilístico rápido y común para clasificación de texto. Funciona como línea base fuerte en NLP.
Logistic Regression	Modelo lineal para clasificación binaria que aprende pesos asociados a cada término del vocabulario.
SVM lineal	Modelo basado en margen, útil en espacios de alta dimensión como los generados por BoW y TF-IDF.

2.6. Experimentos realizados

El pipeline ejecutó doce combinaciones entre procesamiento, representación vectorial y modelo:

Cuadro 6: Diseño experimental

Procesamiento	Vectorización	Modelos
Stemming	BoW	Naive Bayes, Logistic Regression, SVM lineal
Stemming	TF-IDF	Naive Bayes, Logistic Regression, SVM lineal
Lemmatization	BoW	Naive Bayes, Logistic Regression, SVM lineal
Lemmatization	TF-IDF	Naive Bayes, Logistic Regression, SVM lineal

3. Evidencia de ejecución real

El proyecto se ejecutó con el siguiente comando:

```
python -m src.main --data data/raw/kindle_reviews.csv
```

Las siguientes capturas corresponden a la ejecución real con el dataset completo.

```
Dataset cargado: 886484 reseñas
Label
Positivo 829256
Negativo 57148
Name: count, dtype: int64

Preprocesamiento: stemming
Representación: BOW
Naive Bayes: Train Acc=0.9373 | Test Acc=0.9363 | F1=0.9653 | Tiempo=194.5015s
Logistic Regression: Train Acc=0.9723 | Test Acc=0.9698 | F1=0.984 | Tiempo=234.9781s
SVM Lineal: Train Acc=0.9715 | Test Acc=0.9691 | F1=0.9836 | Tiempo=218.8646s
Representación: TFIDF
Naive Bayes: Train Acc=0.9482 | Test Acc=0.9479 | F1=0.9729 | Tiempo=177.671s
Logistic Regression: Train Acc=0.9718 | Test Acc=0.9702 | F1=0.9842 | Tiempo=199.0773s
SVM Lineal: Train Acc=0.9732 | Test Acc=0.9711 | F1=0.9847 | Tiempo=254.298s

Preprocesamiento: lemmatization
Representación: BOW
Naive Bayes: Train Acc=0.9373 | Test Acc=0.9363 | F1=0.9653 | Tiempo=181.8573s
Logistic Regression: Train Acc=0.9723 | Test Acc=0.9698 | F1=0.984 | Tiempo=212.5594s
SVM Lineal: Train Acc=0.9715 | Test Acc=0.9691 | F1=0.9836 | Tiempo=203.5835s
Representación: TFIDF
Naive Bayes: Train Acc=0.9482 | Test Acc=0.9479 | F1=0.9729 | Tiempo=189.2878s
Logistic Regression: Train Acc=0.9718 | Test Acc=0.9702 | F1=0.9842 | Tiempo=206.2924s
SVM Lineal: Train Acc=0.9732 | Test Acc=0.9711 | F1=0.9847 | Tiempo=139.2664s

Resultados guardados en:
/home/other_user/Downloads/Mini_proyecto/sentiment_analysis_nlp_github/outputs/reports/resultados_comparativos.csv
/home/other_user/Downloads/Mini_proyecto/sentiment_analysis_nlp_github/outputs/reports/classification_reports.txt

Mejor configuración:
configuracion representacion procesamiento modelo train accuracy test accuracy precision recall f1 score test_loss tiempo_entrenamiento seg num_documentos
TFIDF + lemmatization tfidf lemmatization SVM Lineal 0.9732 0.9711 0.9779 0.9915 0.9847 NaN 139.2664 (n(mye(myenv(mye(myenv) othe(myenv(myenv) (myenv(n(m(n(m(myenv) other_

python -m src.main --data data/raw/kindle_reviews.csv
Dataset cargado: 886484 reseñas
Label
Positivo 829256
Negativo 57148
Name: count, dtype: int64

Preprocesamiento: stemming
Representación: BOW
Naive Bayes: Train Acc=0.9373 | Test Acc=0.9363 | F1=0.9653 | Tiempo=194.5015s
Logistic Regression: Train Acc=0.9723 | Test Acc=0.9698 | F1=0.984 | Tiempo=234.9781s
SVM Lineal: Train Acc=0.9715 | Test Acc=0.9691 | F1=0.9836 | Tiempo=218.8646s
Representación: TFIDF
Naive Bayes: Train Acc=0.9482 | Test Acc=0.9479 | F1=0.9729 | Tiempo=177.671s
Logistic Regression: Train Acc=0.9718 | Test Acc=0.9702 | F1=0.9842 | Tiempo=199.0773s
SVM Lineal: Train Acc=0.9732 | Test Acc=0.9711 | F1=0.9847 | Tiempo=254.298s

Preprocesamiento: lemmatization
Representación: BOW
Naive Bayes: Train Acc=0.9373 | Test Acc=0.9363 | F1=0.9653 | Tiempo=181.8573s
Logistic Regression: Train Acc=0.9723 | Test Acc=0.9698 | F1=0.984 | Tiempo=212.5594s
SVM Lineal: Train Acc=0.9715 | Test Acc=0.9691 | F1=0.9836 | Tiempo=203.5835s
Representación: TFIDF
(myenv) other_user@other:~/Downloads/Mini_proyecto/sentiment_analysis_nlp_github$ 0.9711 0.9779 0.9915 0.9847 NaN 139.2664 (n(mye(myenv(mye(myenv) othe(myenv(myenv) (myenv(n(m(n(m(myenv) other_
```

Figura 1: Evidencia de ejecución real: carga del dataset, distribución de clases y resultados iniciales.

En la primera evidencia se observa la carga del dataset, el procesamiento de reseñas y el inicio de los resultados por configuración. También se confirma que el pipeline trabajó con el conjunto real y no únicamente con una muestra.

```
(myenv) other_user@other:~/Downloads/Mini_proyecto/sentiment_analysis_nlp_github$ python -m src.main --data data/raw/kindle_reviews.csv
Dataset cargado: 886404 reseñas
Label
Positivo 829256
Negativo 57148
Name: count, dtype: int64

Preprocesamiento: stemming
Representación: BOW
Naive Bayes: Train Acc=0.9373 | Test Acc=0.9363 | F1=0.9653 | Tiempo=194.5015s
Logistic Regression: Train Acc=0.9723 | Test Acc=0.9698 | F1=0.984 | Tiempo=234.9701s
SVM Lineal: Train Acc=0.9715 | Test Acc=0.9691 | F1=0.9836 | Tiempo=210.8646s
Representación: TFIDF
Naive Bayes: Train Acc=0.9402 | Test Acc=0.9479 | F1=0.9729 | Tiempo=177.671s
Logistic Regression: Train Acc=0.9718 | Test Acc=0.9702 | F1=0.9842 | Tiempo=199.0773s
SVM Lineal: Train Acc=0.9732 | Test Acc=0.9711 | F1=0.9847 | Tiempo=254.298s

Preprocesamiento: lemmatization
Representación: BOW
Naive Bayes: Train Acc=0.9373 | Test Acc=0.9363 | F1=0.9653 | Tiempo=181.8573s
Logistic Regression: Train Acc=0.9723 | Test Acc=0.9698 | F1=0.984 | Tiempo=212.5594s
SVM Lineal: Train Acc=0.9715 | Test Acc=0.9691 | F1=0.9836 | Tiempo=203.5835s
Representación: TFIDF
Naive Bayes: Train Acc=0.9402 | Test Acc=0.9479 | F1=0.9729 | Tiempo=189.2878s
Logistic Regression: Train Acc=0.9718 | Test Acc=0.9702 | F1=0.9842 | Tiempo=206.2924s
SVM Lineal: Train Acc=0.9732 | Test Acc=0.9711 | F1=0.9847 | Tiempo=139.2664s

Resultados guardados en:
/home/other_user/Downloads/Mini_proyecto/sentiment_analysis_nlp_github/outputs/reports/resultados_comparativos.csv
/home/other_user/Downloads/Mini_proyecto/sentiment_analysis_nlp_github/outputs/reports/classification_reports.txt

Mejor configuración:
configuracion representation procesamiento modelo train_accuracy test_accuracy precision recall f1 score test_loss tiempo_entrenamiento seg num_documentos
TFIDF - lemmatization tfidf lemmatization SVM Lineal 0.9732 0.9711 0.9779 0.9915 0.9847 NaN 139.2664 (m(myenv(myenv(myenv) othe(myenv(myenv) (myenv(m(m(m(m(myenv) other_
(myenv) other_user@other:~/Downloads/Mini_proyecto/sentiment_analysis_nlp_github$
python -m src.main --data data/raw/kindle_reviews.csv
Dataset cargado: 886404 reseñas
Label
Positivo 829256
Negativo 57148
Name: count, dtype: int64

Preprocesamiento: stemming
Representación: BOW
Naive Bayes: Train Acc=0.9373 | Test Acc=0.9363 | F1=0.9653 | Tiempo=194.5015s
Logistic Regression: Train Acc=0.9723 | Test Acc=0.9698 | F1=0.984 | Tiempo=234.9701s
SVM Lineal: Train Acc=0.9715 | Test Acc=0.9691 | F1=0.9836 | Tiempo=210.8646s
Representación: TFIDF
Naive Bayes: Train Acc=0.9402 | Test Acc=0.9479 | F1=0.9729 | Tiempo=177.671s
Logistic Regression: Train Acc=0.9718 | Test Acc=0.9702 | F1=0.9842 | Tiempo=199.0773s
SVM Lineal: Train Acc=0.9732 | Test Acc=0.9711 | F1=0.9847 | Tiempo=254.298s

Preprocesamiento: lemmatization
Representación: BOW
Naive Bayes: Train Acc=0.9373 | Test Acc=0.9363 | F1=0.9653 | Tiempo=181.8573s
Logistic Regression: Train Acc=0.9723 | Test Acc=0.9698 | F1=0.984 | Tiempo=212.5594s
SVM Lineal: Train Acc=0.9715 | Test Acc=0.9691 | F1=0.9836 | Tiempo=203.5835s
```

Figura 2: Evidencia de ejecución real: cierre de la corrida y mejor configuración obtenida.

En la segunda evidencia se observa el cierre de la ejecución y la identificación de la mejor combinación: TF-IDF, lemmatization y SVM lineal.

4. Resultados obtenidos

Cuadro 7: Resultados reales obtenidos con el dataset completo

Configuración	Modelo	Train Acc	Test Acc	F1-score	Tiempo (s)
BoW + stemming	Naive Bayes	0.9373	0.9363	0.9653	194.5015
BoW + stemming	Logistic Regression	0.9723	0.9698	0.9840	234.9781
BoW + stemming	SVM lineal	0.9715	0.9691	0.9836	210.8646
TF-IDF + stemming	Naive Bayes	0.9482	0.9479	0.9729	177.6710
TF-IDF + stemming	Logistic Regression	0.9718	0.9702	0.9842	199.0773
TF-IDF + stemming	SVM lineal	0.9732	0.9711	0.9847	254.2980
BoW + lemmatization	Naive Bayes	0.9373	0.9363	0.9653	181.8573
BoW + lemmatization	Logistic Regression	0.9723	0.9698	0.9840	212.5594
BoW + lemmatization	SVM lineal	0.9715	0.9691	0.9836	203.5835
TF-IDF + lemmatization	Naive Bayes	0.9482	0.9479	0.9729	189.2878
TF-IDF + lemmatization	Logistic Regression	0.9718	0.9702	0.9842	206.2924
TF-IDF + lemmatization	SVM lineal	0.9732	0.9711	0.9847	139.2664

4.1. Comparación de F1-score

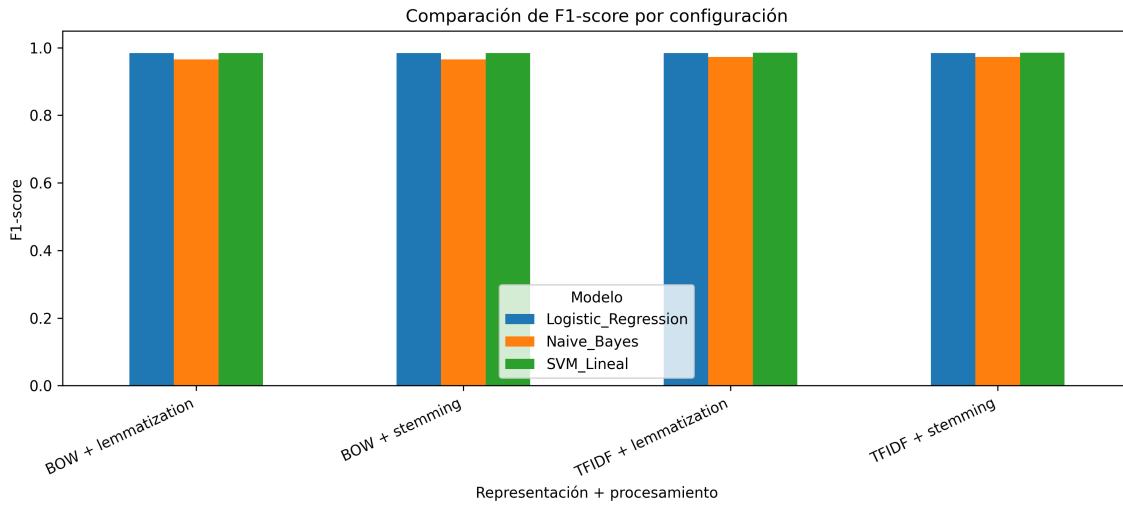


Figura 3: Comparación de F1-score por configuración.

La gráfica muestra que los modelos lineales superan a Naive Bayes en la mayoría de las configuraciones. Logistic Regression y SVM lineal obtienen resultados muy cercanos, pero SVM lineal alcanza el mejor F1-score general de 0.9847.

4.2. Comparación de tiempo de entrenamiento

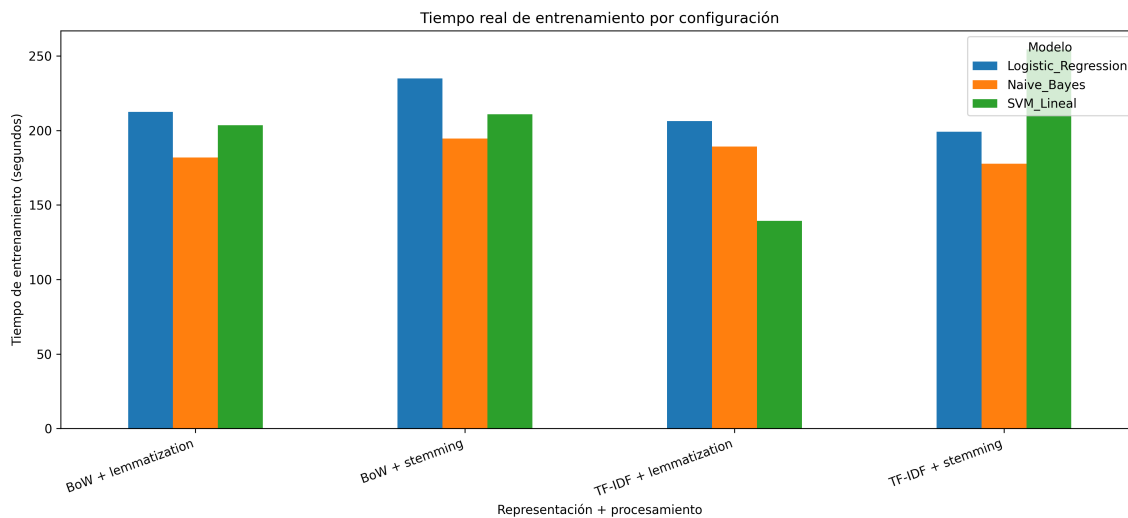


Figura 4: Comparación del tiempo de entrenamiento por configuración.

El tiempo de entrenamiento varía según la representación, el método de procesamiento y el modelo. La mejor configuración, TF-IDF + lemmatization + SVM lineal, tuvo un tiempo competitivo de 139.2664 segundos en la ejecución registrada.

5. Matrices de confusión

Las matrices de confusión permiten revisar cuántas reseñas fueron clasificadas correctamente y cuántas fueron confundidas entre clase positiva y clase negativa. A continuación se incluyen las doce matrices generadas por el pipeline usando los nombres reales de archivo observados.

5.1. Bag of Words con lemmatization

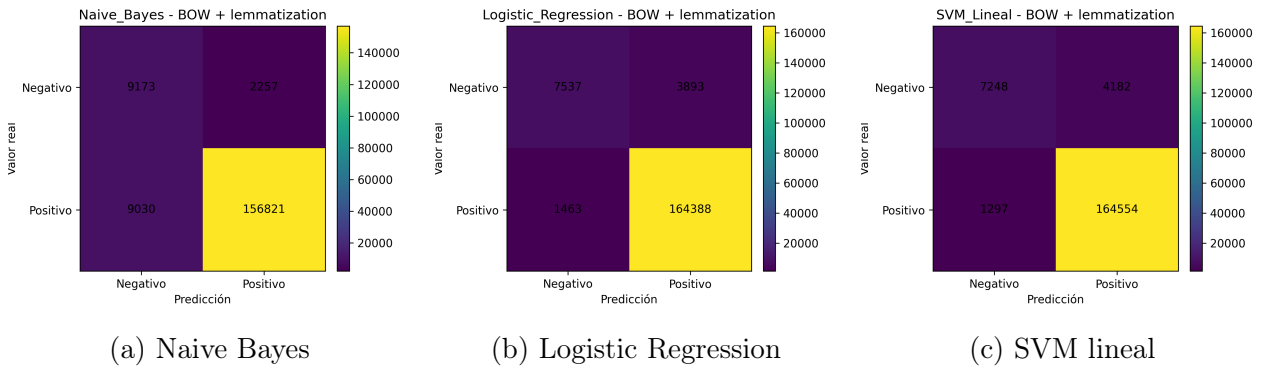


Figura 5: Matrices de confusión para Bag of Words con lemmatization.

5.2. Bag of Words con stemming

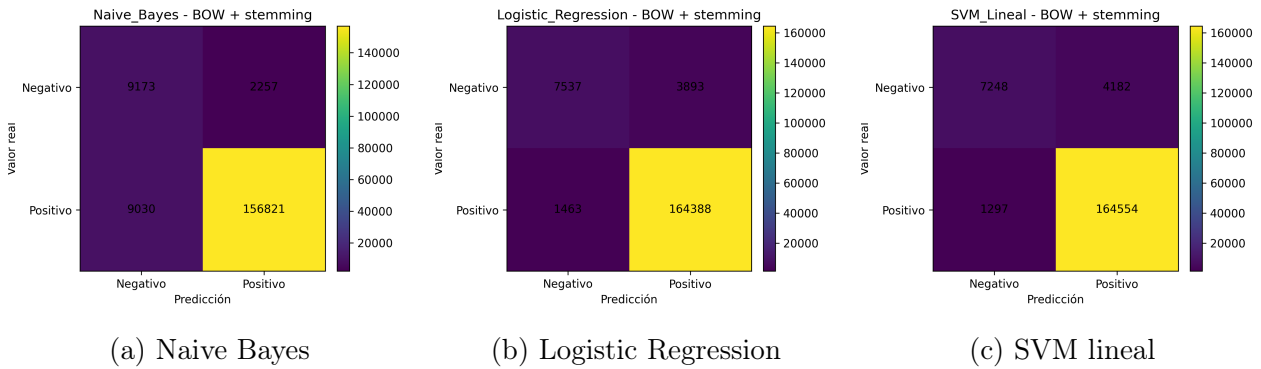


Figura 6: Matrices de confusión para Bag of Words con stemming.

5.3. TF-IDF con lemmatization

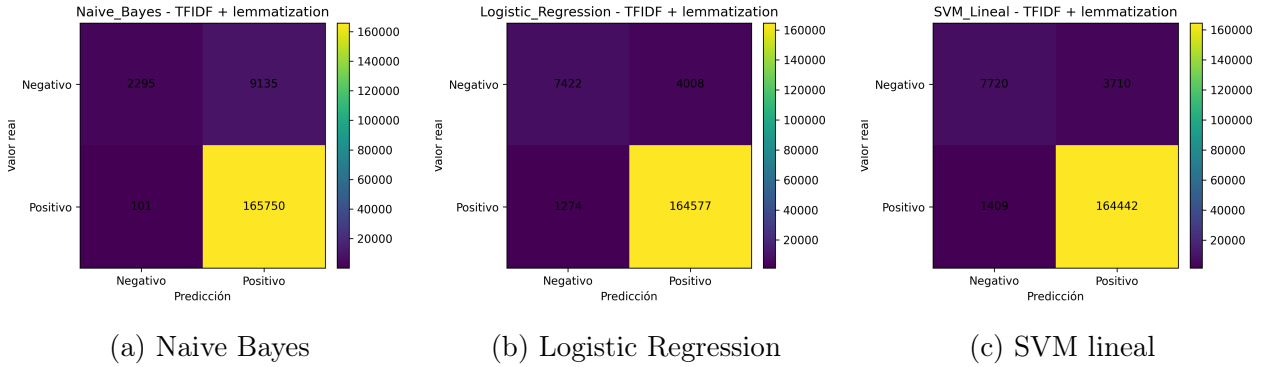


Figura 7: Matrices de confusión para TF-IDF con lemmatization.

5.4. TF-IDF con stemming

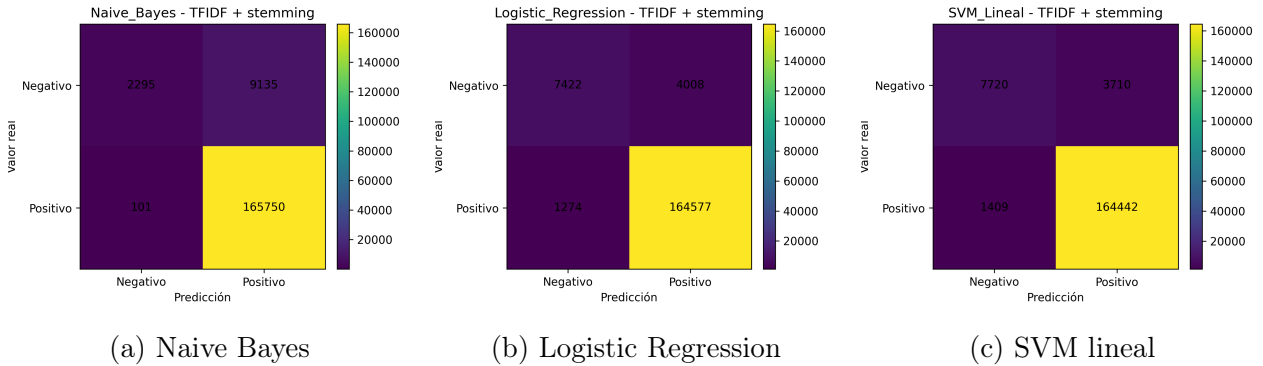


Figura 8: Matrices de confusión para TF-IDF con stemming.

5.5. Interpretación general de las matrices

Las matrices muestran que las combinaciones con modelos lineales reducen más los errores de clasificación que Naive Bayes. Esto es consistente con los resultados de F1-score: Logistic Regression y SVM lineal aprovechan mejor los espacios vectoriales generados por Bag of Words y TF-IDF. La mejor matriz corresponde al modelo SVM lineal con TF-IDF y lemmatization, que coincide con el mejor desempeño global reportado.

6. Análisis comparativo

6.1. Bag of Words vs. TF-IDF

Bag of Words obtuvo resultados altos, pero TF-IDF generó la mejor configuración general. La ventaja de TF-IDF es que no solo cuenta palabras, sino que pondera su importancia. Esto ayuda a reducir el efecto de términos frecuentes que aparecen en muchas reseñas y que no necesariamente aportan información útil para distinguir sentimientos.

6.2. Stemming vs. lemmatization

Con Bag of Words, stemming y lemmatization produjeron resultados prácticamente iguales. Con TF-IDF, ambas técnicas alcanzaron el mismo F1-score máximo con SVM lineal. Sin embargo, la combinación TF-IDF + lemmatization + SVM lineal fue más rápida en la ejecución registrada, con un tiempo de 139.2664 segundos.

6.3. Comparación de modelos

Naive Bayes fue el modelo más simple y obtuvo resultados adecuados, pero quedó por debajo de Logistic Regression y SVM lineal. Logistic Regression tuvo un desempeño fuerte y estable. SVM lineal obtuvo el mejor F1-score, lo cual confirma que es una buena opción para clasificación de texto representado mediante vectores dispersos.

6.4. Problema de desbalance de clases

El principal problema del dataset fue el desbalance. La clase positiva tuvo 829,256 reseñas, mientras que la clase negativa tuvo 57,148. Cuando existe este tipo de desbalance, accuracy puede ser engañoso, porque un modelo podría obtener una exactitud alta prediciendo mayoritariamente la clase dominante. Por esta razón, se revisaron métricas como precision, recall y F1-score.

7. Validación y pruebas automáticas

El proyecto incluye pruebas automáticas que pueden ejecutarse con:

```
pytest -q
```

Las pruebas revisan los siguientes puntos:

- Carga del dataset de muestra.
- Existencia de etiquetas binarias 0 y 1.
- Limpieza básica del texto.
- Salida válida del preprocesamiento.

Además, el pipeline fue validado mediante la ejecución con el dataset real de 886,404 reseñas y mediante la generación de reportes, gráficas comparativas y matrices de confusión.

8. Cómo ejecutar el proyecto

8.1. Crear ambiente virtual

```
python -m venv .venv
```

En Windows:

```
.venv\Scripts\activate
```

En macOS o Linux:

```
source .venv/bin/activate
```

8.2. Instalar dependencias

```
pip install -r requirements.txt
```

8.3. Ejecutar con dataset real

El archivo CSV debe guardarse como:

```
data/raw/kindle_reviews.csv
```

Luego se ejecuta:

```
python -m src.main --data data/raw/kindle_reviews.csv
```

8.4. Ejecutar con dataset de muestra

```
python -m src.main --data data/sample/kindle_reviews_sample.csv
```

El dataset de muestra sirve para verificar que el proyecto corre correctamente, pero no sustituye los resultados obtenidos con el dataset real.

9. Documentación del uso de IA

Se utilizó IA como apoyo para:

- Organizar la estructura del repositorio.
- Proponer funciones auxiliares de carga, limpieza y evaluación.
- Redactar una base inicial de documentación.
- Revisar que el proyecto incluyera los puntos de la rúbrica.

El equipo ejecutó el proyecto, revisó los resultados y validó el comportamiento del pipeline. El porcentaje estimado de código asistido por IA se mantiene por debajo del 30 %.

9.1. Errores encontrados y correcciones

Cuadro 8: Errores encontrados durante el desarrollo y correcciones aplicadas

Error o reto encontrado	Corrección aplicada
El dataset real tarda mucho por su tamaño.	Se limitó el vocabulario con <code>MAX_FEATURES = 5000</code> .
El dataset está desbalanceado.	Se priorizó F1-score sobre accuracy.
La clase neutral genera ambigüedad.	Se eliminaron reseñas con calificación 3.
Algunos modelos no tienen <code>predict_proba</code> .	<code>test_loss</code> queda vacío cuando no aplica.

10. Conclusiones

El proyecto implementa un pipeline completo de *Sentiment Analysis* con NLP. El sistema carga reseñas, limpia texto, transforma palabras en vectores numéricos y entrena modelos supervisados para clasificar sentimientos.

La mejor configuración fue **TF-IDF + lemmatization + SVM lineal**, con **Test Accuracy = 0.9711** y **F1-score = 0.9847**. Este resultado indica que el modelo clasifica correctamente la mayoría de las reseñas y mantiene buen balance entre precision y recall.

TF-IDF funcionó mejor porque pondera términos relevantes y reduce el peso de palabras comunes. SVM lineal fue el mejor modelo porque trabaja bien con vectores dispersos y de alta dimensión. El desbalance de clases fue el reto principal del dataset, por lo que F1-score es una métrica más informativa que accuracy para comparar el desempeño de los modelos.

11. Referencias

Bird, S., Klein, E., & Loper, E. (2009). *Natural language processing with Python*. O'Reilly Media.

Manning, C. D., Raghavan, P., & Schütze, H. (2008). *Introduction to information retrieval*. Cambridge University Press.

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, É. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.

Srivastava, B. (s. f.). *Amazon reviews: Kindle Store Category*. Kaggle. <https://www.kaggle.com/datasets/bharadwaj6/kindle-reviews>